

Looped App: AI Design & Development Prompts

UI/UX Design Prompts (Figma, Uizard, Framer, etc.)

- **Login Screen (Gmail Only):** *Prompt:* "Design the **Looped** login page with the app logo and a Google/Gmail Sign-In button only. No email/password fields. Use a clean, modern mobile layout and brand colors. The Google button should clearly indicate Gmail OAuth."
- **Role Toggle UI:** *Prompt:* "Create a toggle control (e.g. segmented button or switch in the header) labeled **Seeker / Referrer**. When toggled, it switches the view between the Job Seeker dashboard and Referral Provider dashboard. Design both states distinctly so users know which role is active."
- **Job Search & Listing Screen:** *Prompt:* "Design a **Job Listings** screen: include a search bar at the top (placeholder "Search by title or company"), filter icon, and a scrollable list of jobs. Each job card shows title, company, location, and an **Apply for Referral** button. Ensure a clear hierarchy (e.g. bold title, company subtitle)."
- **Referral Request Form:** *Prompt:* "Design a **Referral Request** screen for seekers. It should show the selected job info and a form to upload a resume (file picker UI) and a **Pay Referral Fee** button. Indicate the fee amount and use Razorpay branding on the button. Make sure the upload and payment actions are prominent and clear."
- **Seeker Dashboard:** *Prompt:* "Create a **Seeker Dashboard** view listing the user's referral requests. Divide into sections like 'Active Requests' and 'Past Requests'. For each request, display job title, company, current status (e.g. Pending, Completed), and amount paid. Include a summary (e.g. total spent)."
- **Referrer Dashboard:** *Prompt:* "Design a **Referrer Dashboard** for providers. Show sections: *Assigned Referrals* (pending jobs) and *Completed Referrals*. Each item should show job title, company, status, and earned amount. For each pending referral, include an **Upload Proof** button. At top, display **Total Earnings**."
- **Referral Details Screen:** *Prompt:* "When a user taps a job or referral, show a **Referral Detail** screen: include full job description, company info, and referral status updates. For seekers, include resume and payment info; for referrers, include proof upload status."
- **Navigation Flow:** *Prompt:* "Instruct the AI to create a clickable prototype flow: **Login → Role Toggle → (Seeker Dashboard ↔ Job Search ↔ Referral Request)** and **(Referrer Dashboard → Referral Detail)**. Ensure smooth transitions between screens. Provide annotations for user flows between each component."

Frontend Development Prompts (FlutterFlow, Builder.io, React Native/Copilot, etc.)

- **Project Setup:** *Prompt:* "Initialize a cross-platform mobile project named *Looped*. For example, in FlutterFlow or React Native: set up Firebase authentication (Gmail only) and Razorpay SDK. Include packages: `firebase_auth` and `google_sign_in` (for Flutter) or `@react-native-google-signin/google-signin`; and Razorpay Flutter plugin or `react-native-razorpay`. Structure the app with navigation tabs or drawer."
- **Google (Gmail) OAuth Login:** *Prompt:* "Implement Google Sign-In as the only authentication. In FlutterFlow, use **Log In** action with provider=Google ¹. In code, call `GoogleSignIn` (or

Firebase `signInWithCredential`) on button press. Ensure on success the user's email/name/photo are stored in a **users** collection ¹."

- **Role Switching Logic:** *Prompt:* "Add app state or user profile to support two roles. For example, define a `role` field (values: "seeker" or "referrer") in the user model. Prompt code to create a toggle button: on switch, update the role in state and navigate to the corresponding dashboard. Use a persistent toggle or profile setting so users can switch anytime."
- **Job Search & Listing (Seeker):** *Prompt:* "Build the Job Listings UI with search. Example: Use a `ListView` or `FlatList` to display jobs. On text input change, filter jobs by title/company using Firestore queries or local search. In Flutter, fetch jobs from a `jobs` collection; in React Native, use Firebase SDK or REST API. Ensure fast, real-time updates." ²
- **Referral Request Handling:** *Prompt:* "Implement the referral application flow. After a seeker fills the form (resume upload and fee), prompt code to upload the resume file to cloud storage (Firebase Storage or AWS S3) and create a new **Referral** record (status=pending). Then initiate Razorpay checkout. For example, in Flutter:

```
Razorpay _razorpay = Razorpay();
_razorpay.on(Razorpay.EVENT_PAYMENT_SUCCESS, _handlePaymentSuccess);
// ...
var options = {
  'key': '<YOUR_KEY>',
  'amount': referralFeeInPaise,
  'currency': 'INR',
  'order_id': orderId, // from server
};
_razorpay.open(options);
```

³ ⁴ . On payment success, update the referral status."

- **Referrer Actions (Screenshot Proof):** *Prompt:* "For the assigned referrals list, add functionality to upload proof. When the provider taps **Upload Proof**, prompt code to open image picker and upload the screenshot to storage. Then update the referral record with proof URL and mark status as *completed*. Trigger a notification/email to the seeker if needed."
- **Dashboards Data Binding:** *Prompt:* "Bind data to dashboards. For seeker: query `referrals` where `seekerId = currentUser` and display with statuses. For referrer: query `referrals` where `referrerId = currentUser`. Include earnings summary by summing completed referrals. In FlutterFlow or Builder.io, add these as data queries to `ListView` components."
- **UI Components & Styling:** *Prompt:* "Use native look-and-feel components. For Flutter, use Material widgets (e.g. `AppBar`, `ListTile`, `TabBar`). For React Native, use libraries like React Navigation and UI kits. Ensure accessibility (labels, touch targets) and responsive layout for both Android and iOS."

Backend & Admin Logic Prompts (Firebase / Node.js + MongoDB)

- **Data Models:** *Prompt:* "Define schemas/collections for **Users**, **Jobs**, and **Referrals**. Example (MongoDB/Firestore):

```
User: { id, name, email, company, roles: ['seeker', 'referrer'],
earnings, active }
Job: { id, title, company, description, ... }
```

```
Referral: { id, jobId, seekerId, referrerId, status, proofUrl, amount }
```

Use schema/Firestore rules as needed."

- **Job Posting API/Service:** Prompt: "Set up an API endpoint or Cloud Function to manage job listings. E.g., a protected admin route to add jobs. For search, allow frontend to query jobs by title or company using database indexes."
- **Referral Request API:** Prompt: "Create a backend function triggered when a seeker creates a referral request. It should: (1) validate the request, (2) create a `Referral` record with status 'pending', (3) call the **assignment logic** to pick a referrer."
- **Assignment Logic (Admin):** Prompt: "Implement the assignment algorithm. For example (pseudocode):

```
// Find eligible referrers at same company, who are active
const candidates = db.users.find({ company: job.company, active: true });
// Sort by lowest earnings (new users first)
candidates.sort((a,b) => a.earnings - b.earnings);
// Assign the first candidate
const chosen = candidates[0];
referral.referrerId = chosen.id;
```

Use a transaction or atomic update. Avoid users who are inactive or previously overloaded (e.g. skip if `active=false`). This can run in a Firebase Cloud Function or Node/Express API. Firebase's serverless model is ideal for this logic ²."

- **Database & Scalability:** Prompt: "Choose backend tech for scale. Example prompt: "Use Firebase Firestore (auto-scaling, real-time sync, serverless backend ²) or a Node.js server with MongoDB (flexible schema, high performance ⁵)." Create indexes on `company` and `earnings` for fast queries."
- **Gmail Auth & Users Storage:** Prompt: "Ensure every Google login creates/updates a user record. (FlutterFlow does this with **Create User Document** ¹.) Include fields for earnings and status. For multiple roles, store a roles array or flags."
- **Testing & Security:** Prompt: "Prompt code tools to add input validation and security rules. For Firebase: write Firestore rules so seekers can only create referrals for themselves, and referrers can only update referrals assigned to them. In Node.js: use middleware to authenticate and authorize requests."

Integration Snippets & Features

- **Google Sign-In (FlutterFlow / React Native):** Prompt: "Insert Gmail OAuth. Example (Flutter):

```
final GoogleSignInAccount googleUser = await GoogleSignIn().signIn();
final googleAuth = await googleUser.authentication;
final credential = GoogleAuthProvider.credential(
  idToken: googleAuth.idToken, accessToken: googleAuth.accessToken);
await FirebaseAuth.instance.signInWithCredential(credential);
```

Then create user doc (email, name) in `users` ¹."

- **Razorpay Checkout (Flutter / React Native):** Prompt: "Integrate Razorpay. Example (Flutter with razorpay_flutter):"

```
_razorpay = Razorpay();
_razorpay.on(Razorpay.EVENT_PAYMENT_SUCCESS, _handlePaymentSuccess);
_razorpay.on(Razorpay.EVENT_PAYMENT_ERROR, _handlePaymentError);
var options = {
  'key': 'YOUR_KEY_ID', 'amount': feeInPaise, 'currency': 'INR',
  'order_id': '<GENERATED_ORDER_ID>',
  'name': 'Looped Referral',
  'description': 'Job Referral Fee'
};
_razorpay.open(options);
```

Store the order_id on server and verify payment signature after success ³ ⁴ ."

- **Role Toggle UI (Code):** Prompt: "Implement a role switcher. For example, use a `TabBar` or toggle button:

```
TabBar(tabs: [Text('Seeker'), Text('Referrer')], onTap: (index) {
  setState(() => isSeeker = index==0); });
```

Conditionally show either the Seeker or Referrer dashboard based on this state. Ensure persistence (e.g. in user profile)."

- **Backend Notification (Optional):** Prompt: "Add notifications or emails: when a referrer uploads proof and marks referral completed, notify the seeker. This can be done via Firebase Cloud Messaging or an email trigger."

Sources: Integration steps and best practices are based on official docs. For example, FlutterFlow's Google OAuth guide ¹ and Razorpay's Flutter integration guide ³ ⁴ were used as references. Tech stack recommendations (Flutter UI, Firebase/MongoDB backend) are informed by performance and scalability analyses ⁶ ² ⁵ .

¹ Google OAuth Login | FlutterFlow Documentation

<https://docs.flutterflow.io/integrations/authentication/firebase/google-oauth-login/>

² ⁵ Firebase vs MongoDB: Battle of The Best Databases for 2024 | by iTechnolabs Inc | Medium

<https://itechnolabsca.medium.com/firebase-vs-mongodb-battle-of-the-best-databases-for-2024-93bd2fa761a2>

³ ⁴ Payment Gateway | Flutter - Build Integration | Razorpay Docs

<https://razorpay.com/docs/payments/payment-gateway/flutter-integration/standard/build-integration/?preferred-country=US>

⁶ Flutter vs. React Native in 2025

<https://www.nomtek.com/blog/flutter-vs-react-native>